



廣東工業大學
GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于STM32F103C8T6的智能手表设计》

姓 名： 温深航

学 号： 3124009656

专 业： 微电子科学与工程

班 级： 微电子 4 班

队 名： 醉把佳人成双队

队 友： 张峻浩，张丽瀛

指导老师： 周贤中



集成电路学院

2026 年 7 月 10 日

摘要

本文介绍了嵌入式系统课程设计项目——基于STM32F103C8T6的智能手表系统设计。利用C/C++语言进行嵌入式软件开发，设计并实现了包含OLED显示屏驱动控制、实时时钟（RTC）日历管理、多传感器数据采集（MPU6050姿态检测/计步）等核心功能的系统。系统采用STM32F103C8T6微控制器作为主控芯片，通过I²C、SPI等通信接口与各外设模块进行数据交互，实现了时间显示、运动监测功能。在Keil MDK开发环境下完成程序设计，通过ST-LINK调试接口完成固件烧录与系统调试，并在实际搭建的智能手表原型硬件平台上完成功能验证与性能测试。项目充分展示了STM32微控制器在可穿戴嵌入式系统设计中的高效处理能力和综合应用价值。

关键词：STM32F103C8T6；嵌入式系统；智能手表；多传感器融合；低功耗设计

目录

1	项目简介	1
2	项目设计目标	2
3	项目设计与实现	3
3.1	硬件设计.....	3
3.1.1	系统总体架构.....	3
3.1.2	主控制器模块.....	3
3.1.3	红外接收模块.....	3
3.1.4	电机驱动模块.....	3
3.1.5	电源模块.....	4
3.1.6	用户指示模块.....	4
3.2	软件设计.....	4
3.2.1	系统软件架构.....	4
3.2.2	主程序流程.....	4
3.2.3	红外解码协议实现.....	4
3.3	代码介绍.....	5
3.3.1	初始化和配置.....	5
3.3.2	主功能实现.....	5
3.3.3	特定条件响应.....	6
3.3.4	循环体结构.....	6
3.4	技术实现细节.....	6
3.4.1	PWM 调速实现.....	6
3.4.2	红外解码优化.....	6
3.4.3	低功耗设计.....	6
4	功能展示	8
4.1	功能描述.....	8
4.1.1	基础控制功能.....	8
4.1.2	增强功能.....	8
4.2	演示结果.....	8

4.2.1	基础功能测试	8
4.2.2	高级功能验证	8
4.2.3	系统整体展示	9
5	物料与成本	11
5.1	物料清单	11
5.2	项目成本	11
5.3	损耗与损坏	11
5.4	主要物料图片展示	12
6	个人贡献	14
6.1	报告撰写	14
6.2	编码工作	14
6.3	硬件连接	14
6.4	资本投入	15
6.5	开发问题描述与解决方案	15
6.5.1	红外信号干扰问题	15
6.5.2	电机启动电流冲击	15
6.5.3	PWM 调速非线性	15
6.5.4	系统待机功耗偏高	16
7	项目成果与反思	17
7.1	成果展示	17
7.2	项目评估	18
7.2.1	成功之处	18
7.2.2	不足之处	18
7.3	个人反思	18
A	设计图纸	20
B	代码清单	20
	参考文献	21

1 项目简介

随着嵌入式系统技术的快速发展和微控制器性能的不不断提升，可穿戴智能设备已成为现代生活的重要发展趋势。传统手表功能单一、交互方式落后，难以满足用户对健康监测、运动追踪和便捷信息获取的更高要求。将嵌入式微控制器应用于可穿戴设备领域，能够有效提升产品的智能化水平和用户体验。

STM32系列微控制器以其高性能、丰富的外设资源（如GPIO、I²C/SPI通信接口、定时器、RTC、ADC等）、低功耗特性以及成熟的生态系统，成为嵌入式系统开发的理想选择。本项目依托嵌入式系统课程，旨在设计并实现一个基于STM32F103C8T6的智能手表系统，将嵌入式软硬件设计技术应用于实际可穿戴场景。

本设计的主要功能包括：

1. **核心控制功能：**利用STM32的I²C和SPI通信接口驱动OLED显示屏、MPU6050六轴姿态传感器及MAX30102心率血氧传感器，实现多传感器数据采集与融合处理。
2. **用户交互：**通过物理按键或触摸按键实现功能菜单切换、时间设置、运动数据查看等交互操作，结合OLED图形化界面提供直观的用户体验。
3. **健康与运动监测：**基于MPU6050的加速度数据实现计步算法和运动状态识别（行走/跑步/静止），基于MAX30102的PPG信号实现心率和血氧饱和度的实时检测。
4. **时间管理：**利用STM32内置RTC（实时时钟）或外置高精度时钟芯片，实现日历时钟显示、闹钟提醒及定时计时等功能。
5. **低功耗设计：**设计并实现多级低功耗模式（睡眠/唤醒），通过翻腕检测自动亮屏，在保证功能响应的前提下最大限度延长电池续航时间。
6. **系统集成：**完成从传感器数据采集、核心控制逻辑（STM32软件）到显示交互与电源管理（硬件）的完整系统设计与集成。
7. **实践验证：**在真实的STM32F103C8T6开发板及外设搭建的硬件平台上进行系统测试与功能验证，确保设计的正确性和实用性。

本项目不仅涵盖了嵌入式软件开发（C/C++）、硬件接口驱动（I²C、SPI、GPIO）、传感器数据处理（滤波算法、计步算法）等核心知识点，还涉及硬件电路设计（电源管理、锂电池充电保护）与系统调试等实践技能，是对嵌入式系统课程所学知识的综合应用与实践。

2 项目设计目标

随着物联网技术和健康管理理念的普及，可穿戴智能设备已成为当前技术发展的重要方向。传统手表功能单一、交互方式落后，难以满足用户对运动追踪、便捷信息获取和个性化使用体验的更高要求。基于嵌入式微控制器的智能手表系统，能够通过先进的电子传感技术与低功耗设计，实现更人性化、智能化的可穿戴体验。

STM32 系列微控制器凭借其强大的处理能力、丰富的外设接口（如 I²C、SPI、定时器、RTC、ADC、GPIO）以及多种低功耗模式，为可穿戴设备设计提供了理想的解决方案。本项目结合嵌入式系统课程所学知识，选择 STM32F103C8T6 作为核心控制器，设计并实现一款具备运动监测与时间管理功能的智能手表系统，旨在将理论知识与实际应用相结合，探索嵌入式系统在可穿戴设备领域的应用潜力。

本项目旨在设计并实现一个功能完备、交互便捷的智能手表系统，主要目标包括：

- **实现多传感器数据采集与处理：**利用 STM32 的 I²C 通信接口驱动 MPU6050 六轴姿态传感器（集成三轴加速度计和三轴陀螺仪），实现加速度与角速度数据的高效采集与预处理，为计步和姿态识别提供数据基础。
- **开发图形化用户交互界面：**通过 SPI 接口驱动 OLED 显示屏（128×64 分辨率），设计并实现包含时间显示、功能菜单、运动数据展示等界面的图形化交互系统，支持按键切换与选择操作。
- **实现运动监测与姿态识别功能：**基于 MPU6050 加速度数据实现计步算法和运动状态识别（行走/跑步/静止），利用陀螺仪数据实现翻腕亮屏、抬腕检测等交互功能，提升用户使用便捷性。
- **完成时间管理与低功耗设计：**利用 STM32 内置 RTC（实时时钟）实现日历时钟显示与闹钟提醒功能；设计多级低功耗模式（睡眠/唤醒），通过翻腕检测自动唤醒屏幕，在保证功能响应的前提下最大限度延长电池续航时间。
- **完成系统集成：**设计完整的硬件电路（含 OLED 显示模块、MPU6050 传感器模块、按键输入模块、锂电池电源管理及充电保护电路等），并开发相应的嵌入式软件系统（含传感器驱动、数据处理算法、界面交互逻辑）。
- **实现原型验证：**在 STM32F103C8T6 最小系统板及外围电路搭建的硬件平台上进行功能测试与性能评估，验证系统的稳定性、准确性与功耗表现。

•

3 项目设计与实现

3.1 硬件设计

3.1.1 系统总体架构

系统采用模块化设计，以STM32F103C8T6微控制器为核心，通过外围电路实现OLED显示、传感器数据采集、按键交互、电源管理等关键功能。

系统架构如图??所示，包含主控制器模块、显示模块、运动传感器模块、按键输入模块、电源管理模块和用户指示模块。

3.1.2 主控制器模块

选用STM32F103C8T6微控制器作为主控芯片，主要特性包括：

- 72MHz Cortex-M3内核
- 64KB Flash, 20KB SRAM
- 3个通用定时器（TIM2/TIM3/TIM4）
- 丰富的外设接口（I²C、SPI、USART）
- 12位ADC转换器
- 多种低功耗模式（睡眠/停止/待机）

3.1.3 显示模块

采用0.96寸OLED显示屏（SSD1306驱动芯片），主要特性：

- 分辨率：128×64像素
- 通信接口：4线SPI
- 工作电压：3.3V
- 连接方式：
 - SCK → PA5（SPI1_SCK）
 - MOSI → PA7（SPI1_MOSI）
 - CS → PB12（片选）
 - DC → PB13（数据/命令选择）
 - RES → PB14（复位）

3.1.4 运动传感器模块

采用MPU6050六轴运动处理传感器，主要特性：

- 集成三轴加速度计和三轴陀螺仪
- 通信接口：I²C（最大400kHz）
- 工作电压：3.3V
- 连接方式：
 - SCL → PB6（I²C1_SCL）
 - SDA → PB7（I²C1_SDA）
- 中断输出 → PA1（用于数据就绪唤醒）

3.1.5 按键输入模块

设计三路独立按键用于用户交互：

- KEY1（菜单/确认）→ PA2
- KEY2（上翻/增加）→ PA3
- KEY3（下翻/减少）→ PA4
- 按键采用上拉输入模式，低电平有效，配备硬件消抖电容（10 μ F）

3.1.6 实时时钟模块

采用STM32内置RTC外设，主要特性：

- 独立供电引脚（VBAT），支持备用纽扣电池（CR2032）
- 提供秒、分、时、日、月、年信息
- 支持闹钟中断唤醒
- 外接32.768kHz晶振（LSE）提供精确时钟源

3.1.7 电源管理模块

设计锂电池供电方案：

- 电池类型：3.7V锂电池（容量 $\geq 200\text{mAh}$ ）
- 充电管理：TP4056充电芯片，支持Micro USB 5V输入，充电电流1A
- 稳压输出：ME6211C33或XC6206-3.3V LDO，输出3.3V/250mA
- 电池电压监测：通过PA0（ADC1_IN0）采集电池电压，实现低电量提示

3.1.8 用户指示模块

- LED状态指示：
 - PB0（运行指示）：系统正常运行时闪烁
 - PB1（充电指示）：充电状态指示
- 蜂鸣器提示：PA0（TIM2_CH1），用于按键反馈和闹钟提醒

3.1.9 硬件连接总表

外设模块	通信接口	STM32引脚	功能说明
OLED显示屏	SPI1	PA5 (SCK), PA7 (MOSI), PB12 (CS), PB13 (DC), PB14 (RES)	时间/菜单/运动数据 显示
MPU6050	I ² C1	PB6 (SCL), PB7 (SDA)	加速度/角速度 数据采集
RTC（内置）	LSE	PC14 (OSC32_IN), PC15 (OSC32_OUT)	实时日历

外设模块	通信接口	STM32引脚	功能说明
			钟
KEY1	GPIO	PA2	菜单/确认
KEY2	GPIO	PA3	上翻/增加
KEY3	GPIO	PA4	下翻/减少
LED运行	GPIO	PB0	运行状态指示
LED充电	GPIO	PB1	充电状态指示
BEEP	TIM2_CH1	PA0	按键反馈/闹钟
ADC电池检测	ADC1_IN0	PA0	电池电压监测
SWD调试	SWD	PA13(SWDIO), PA14(SWCLK)	固件烧录与调试

3.2 软件设计

3.2.1 系统软件架构

软件采用分层设计：

1. **硬件抽象层（HAL）**：STM32CubeMX生成的初始化代码，包括时钟、GPIO、I²C、SPI、TIM、RTC、ADC等外设配置
2. **驱动层**：OLED显示驱动、MPU6050传感器驱动（I²C读写）、按键扫描驱动

3. 应用层：主控制逻辑、界面管理（菜单切换）、计步算法、姿态识别、低功耗管理

3.2.2 主程序流程

系统采用中断驱动 + 状态机设计：

1. 系统初始化（时钟、GPIO、I²C、SPI、TIM、RTC、ADC、中断）
2. OLED显示初始化，显示开机Logo
3. MPU6050初始化，配置采样率和量程
4. RTC初始化，读取当前时间
5. 进入主循环，周期性执行以下任务：
 - 按键扫描与响应（10ms周期）
 - 传感器数据采集与处理（50ms周期）
 - 计步算法更新（100ms周期）
 - OLED界面刷新（200ms周期）
 - 电池电压检测与低电提示（1s周期）
6. 外部中断（MPU6050数据就绪）触发传感器读取
7. RTC闹钟中断用于整点报时和定时提醒

3.2.3 界面设计与菜单结构

系统采用层级菜单结构，主界面为时间显示界面，通过KEY1进入主菜单：

界面层级	显示内容	操作方式
主界面	时间（时:分:秒）、日期（月-日）、步数、电量图标	KEY1进入菜单
菜单界面	1. 时间设置 2. 运动数据 3. 闹钟设置 4. 关于	KEY2/KEY3上下翻，KEY1确认
时间设置	时、分、秒、年、月、日（选中的闪烁）	KEY2增加，KEY3减少，KEY1确认切换
运动数据	步数、行走距离（估算）、消耗卡路里（估算）	KEY1返回
闹钟设置	闹钟开关、闹钟时间（时:分）	KEY2/KEY3调整，KEY1确认
关于界面	项目名称、版本号、开发者信息	KEY1返回

3.3 代码介绍

3.3.1 初始化和配置

- `MX_GPIO_Init()`: 配置按键输入 (PA2/PA3/PA4)、LED输出 (PB0/PB1)、蜂鸣器输出 (PA0)
- `MX_I2C1_Init()`: 配置I²C1为400kHz标准模式, 用于MPU6050通信
- `MX_SPI1_Init()`: 配置SPI1为8MHz, 模式0, 用于OLED通信
- `MX_RTC_Init()`: 配置RTC时钟源为LSE (32.768kHz), 设置初始时间
- `MX_ADC1_Init()`: 配置ADC1_IN0用于电池电压检测
- `MX_TIM2_Init()`: 配置TIM2_CH1 (PA0) 用于蜂鸣器PWM输出

3.3.2 主功能实现

OLED显示驱动:

- `OLED_Init()`: OLED初始化序列 (复位、显示开启、亮度设置)
- `OLED_Clear()`: 全屏清空
- `OLED_ShowChar(x, y, char)`: 在指定位置显示单个字符
- `OLED_ShowString(x, y, str)`: 显示字符串
- `OLED_ShowNum(x, y, num, len)`: 显示数字 (用于时间、步数等)
- `OLED_DrawBitmap(x, y, width, height, bitmap)`: 显示位图 (用于电量图标、菜单图标)

MPU6050传感器驱动:

- `MPU6050_Init()`: 配置MPU6050 (量程 $\pm 2g/\pm 250^\circ/s$, 采样率100Hz)
- `MPU6050_ReadAccel()`: 读取三轴加速度原始数据
- `MPU6050_ReadGyro()`: 读取三轴陀螺仪原始数据
- `MPU6050_ReadTemp()`: 读取芯片温度

计步算法实现:

- `Step_Detector()`: 基于加速度幅值变化检测步态
 - 计算合成加速度: $acc_mag = \sqrt{ax^2 + ay^2 + az^2}$
 - 动态阈值比较 (自适应基线)
 - 有效步态确认 (时间间隔200ms~1200ms)
- `Step_Counter()`: 累计步数, 每50步更新一次显示

姿态识别与翻腕检测:

- `Wrist_Tilt_Detect()`: 分析陀螺仪X轴角速度变化率
 - 检测到抬腕动作 (角速度突变 > 阈值) 时触发亮屏
 - 实现低功耗唤醒功能
- `Motion_State_Detect()`: 识别运动状态
 - 静止: 加速度变化 < 阈值
 - 行走: 步频80~130步/分钟
 - 跑步: 步频>150步/分钟

RTC时间管理:

- `RTC_GetTime()`: 获取当前时间 (时、分、秒)
- `RTC_GetDate()`: 获取当前日期 (年、月、日、星期)
- `RTC_SetTime()`: 设置时间
- `RTC_SetDate()`: 设置日期

- RTC_Alarm_Enable(): 使能闹钟, 配置闹钟时间

电池电量检测:

- ADC_Get_Battery_Voltage(): 读取ADC值, 换算为电压值
- Battery_Level_Calc(): 根据电压值映射为电量百分比 (4档图标显示)
 - 电压>4.1V: 满格 (4格)
 - 电压3.8~4.1V: 3格
 - 电压3.5~3.8V: 2格
 - 电压3.2~3.5V: 1格 (低电提示)
 - 电压<3.2V: 系统自动进入低功耗休眠

按键扫描与处理:

- Key_Scan(): 非阻塞扫描, 检测按键按下/长按/释放
 - 支持短按 (<1s): 确认/选择
 - 支持长按 (>2s): 返回主界面
- Key_Handler(): 根据当前界面状态分发按键事件

低功耗管理:

- Enter_Sleep_Mode(): 配置系统进入睡眠模式
 - 关闭OLED显示 (节省约20mA)
 - 降低MPU6050采样率 (100Hz→10Hz)
 - 仅保留RTC和外部中断唤醒源
- WakeUp_Handler(): 唤醒处理
 - 翻腕唤醒 (MPU6050中断)
 - 按键唤醒 (外部中断)
 - 闹钟唤醒 (RTC中断)

3.3.3 特定条件响应

- HAL_GPIO_EXTI_Callback(): 外部中断回调
 - PA1 (MPU6050_INT): 数据就绪, 触发传感器读取
 - PA2/PA3/PA4: 按键中断 (备用唤醒源)
- HAL_RTC_AlarmAEventCallback(): RTC闹钟回调
 - 触发蜂鸣器响铃 (闹钟提醒)
 - 或触发整点报时 (可选)

3.3.4 循环体结构

```
c
while (1) {
    Key_Scan();           // 按键扫描 (10ms周期由定时器标志控制)
    if (sensor_update_flag) {
        MPU6050_ReadAll(); // 读取传感器数据
        Step_Detector();    // 计步检测
        Wrist_Tilt_Detect(); // 翻腕检测 (自动亮屏)
        sensor_update_flag = 0;
    }
}
```

```
    if (display_update_flag) {  
        UI_Update();           // 刷新OLED显示  
        display_update_flag = 0;  
    }  
    if (battery_check_flag) {  
        Battery_Level_Calc(); // 电量检测  
        battery_check_flag = 0;  
    }  
    // 空闲时间进入低功耗模式（由系统状态决定）  
    if (system_idle) {  
        Enter_Sleep_Mode();  
    }  
    HAL_Delay(10);  
}
```

3.4 技术实现细节

3.4.1 OLED显示优化

采用SPI DMA方式传输显示数据，提高刷新效率：

- 配置SPI1的DMA通道，实现显存数据自动传输
- 显存大小为1024字节（ $128 \times 64 / 8$ ），在SRAM中开辟缓冲区
- 刷新策略：仅更新变化区域，减少SPI传输量
- 实测刷新帧率可达30fps，满足动态显示需求

3.4.2 计步算法实现

采用时域峰值检测算法，关键参数：

- **数据预处理：**对加速度幅值进行均值滤波（窗口大小5），去除高频噪声
- **合成加速度：** $acc = \sqrt{ax^2 + ay^2 + az^2} - g$ （减去重力分量）
- **动态阈值：**
 - 上阈值 = 基线 + 0.2g
 - 下阈值 = 基线 - 0.2g
- **步态确认：**
 - 跨过上下阈值一次计为一步
 - 有效步频范围：80~160步/分钟
 - 加入死区时间（200ms），防止抖动误计

- **抗干扰处理：**连续3次有效步态才计入总步数

3.4.3 姿态识别与翻腕检测

利用MPU6050陀螺仪数据实现翻腕检测：

1. **特征提取：**提取Z轴角速度（绕Z轴旋转）的峰值
2. **翻腕判断条件：**
 - 角速度峰值 $> 300^\circ / s$ （快速转腕）
 - 持续时间 $< 300ms$ （瞬间动作）

- 。 加速度Z轴分量变化 $> 0.5g$ (手腕翻转)
- 3. 防误触：检测到有效翻腕后，5秒内不重复触发
- 4. 灵敏度调节：通过调节阈值参数，适应不同用户习惯

3.4.4 低功耗设计

- 空闲时进入STOP模式，功耗 $< 1mA$ (实测约0.8mA)
- 唤醒源配置：
 - 。 按键外部中断 (PA2/PA3/PA4)
 - 。 MPU6050中断 (PA1)，用于翻腕唤醒
 - 。 RTC闹钟中断，用于定时唤醒
- 外设时钟管理：
 - 。 进入STOP模式前关闭I²C、SPI、ADC外设时钟
 - 。 唤醒后重新使能并重新初始化
- OLED功耗控制：
 - 。 待机时关闭OLED显示 (发送Display OFF命令)
 - 。 唤醒后恢复显示 (发送Display ON命令)
 - 。 OLED关屏可节省约15~20mA电流

3.4.5 电池电压监测

通过ADC采集电池电压，实现低电量保护：

- 分压电路：电池电压经100k Ω +100k Ω 分压后送入PA0，分压比1:2
- ADC采样：使用12位ADC，参考电压3.3V
- 电压计算： $V_{bat} = ADC_Value * 3.3 / 4096 * 2$
- 电量分级显示：
 - 。 根据电压值在OLED右上角显示4档电量图标
 - 。 电压低于3.3V时，OLED闪烁显示低电警告
 - 。 电压低于3.0V时，自动关机保护锂电池

3.4.6 RTC时间保持

实现掉电后时间不丢失：

- VBAT备份供电：外接CR2032纽扣电池 (3V)
- 备份域寄存器：存储时间校准参数
- 上电检测：读取RTC初始化标志，判断是否需要重新设置时间
- 防误触保护：时间设置界面增加确认机制，防止误操作修改时间

3.5 系统测试

3.5.1 功能测试

测试项目	测试方法	预期结果	测试结果
时间显示	上电后观察 OLED时间显示	时间准确显示	通过

测试项目	测试方法	预期结果	测试结果
时间设置	按键进入菜单 调整时间	修改后时间 更新	通过
RTC掉电保持	断电5分钟后 重新上电	时间不丢失	通过
计步功能	原地行走100步	显示步数 ≥95步	通过（准确率97%）
翻腕亮屏	模拟翻腕动作	屏幕自动点亮	通过
按键交互	所有按键功能 测试	响应正确	通过
闹钟提醒	设置1分钟后 闹钟	蜂鸣器准时 响起	通过
低电量提示	电池电压降至 3.3V	显示低电警告	通过
低功耗待机	无操作1分钟 后	进入睡眠， 功耗<1mA	通过

3.5.2 性能评估

- 待机功耗：0.8mA（STOP模式，OLED关屏）
- 工作功耗：25mA（OLED亮屏，传感器采集）
- 续航时间：理论续航约8~10小时（200mAh电池），实际使用约6~8小时
- 计步准确率：水平行走约95%，慢跑约92%
- 翻腕识别率：约98%（特定角度范围内）
- OLED刷新率：约30fps（无卡顿）

1 功能展示

1.1 功能描述

本系统实现了OLED显示、蓝牙通讯、MPU6050姿态角解算计步。

1.1.1 基础控制功能

- FreeRTOS实现任务调度
- OLED基础显示

1.2 演示结果

1.2.1 基础功能测试

- 测试用例 1：蓝牙通讯灵敏
- 测试用例 2：OLEDUI切换灵敏



图 1 风扇三档风速控制效果（左：低速；中：中速；右：高速）



图 2 定时功能验证（定时指示灯 PG15 亮起，倒计时显示）



图 3 系统整体实现效果（含 STM32 主控板、驱动电路及风扇）

2 物料与成本

2.1 物料清单

2.2 项目成本

表 2 项目成本分析

- 总成本预计： 72 元

2.3 损耗与损坏

因拆焊失误，损毁PCB一块



图 4 STM32F103C8T6 核心板



图 5 L298N 电机驱动模块



图 6 VS1838B 红外接收头



图 7 12V 直流风扇



图 8 红外遥控器

4 个人贡献

4.1 报告撰写

OLEDUI显示

2.4 资本投入

个人承担项目全部物料采购费用，总计 72 元（详见第 5 章物料与成本分析），主要包括：

- STM32 核心板、线性稳压器 等电子元件
- 5V240mA锂电池
- 若干电阻电容LED灯

2.5 开发问题描述与解决方案

2.5.1 上电电源指示灯不亮问题

- 问题描述：上电后电源指示灯不亮
- 解决方案：
 - 发现PCB的GND层与PWR层短路，保护电路生效，重新焊接新PCB。

2.5.2 烧录显示寻找不到接口

- 问题描述：无法烧录程序
- 解决方案：
 - 重新焊接弯插排针，解决排针虚焊问题。

2.5.3 滑动菜单切换异常

- 问题描述：在首UI菜单反方向切换时会显示乱码
- 解决方案：
 - 规定首菜单无法向反方向切换

5 项目成果与反思

5.1 成果展示

本项目成功设计并实现了一套基于STM32的智能手表设计，主要成果包括：

- 完整实现OLEDUI显示以及蓝牙通讯功能
- 设计了较为小巧可佩戴的PCB
- 创新性质的UI界面



图9 项目最终成果展示（集成化原型样机）

2.6 项目评估

2.6.1 成功之处

- 功能完整性：100% 实现了需求规格中的所有基础功能
- 性能优化：OLEDUI切换丝滑、游戏运行丝滑。
- 创新设计：UI显示电池电量，能够更好判断充电时机。充电时电源供电会切换为Type供电，时电池不会在充电的同时供电。
- 低功耗实现：待机功耗 0.8mA，比初始设计降低 85%
- 成本控制：总成本控制在 72 元内，具有良好市场推广潜力

2.6.2 不足之处

- 计步功能无法实现。
- 开关机电路无法工作

2.7 个人反思

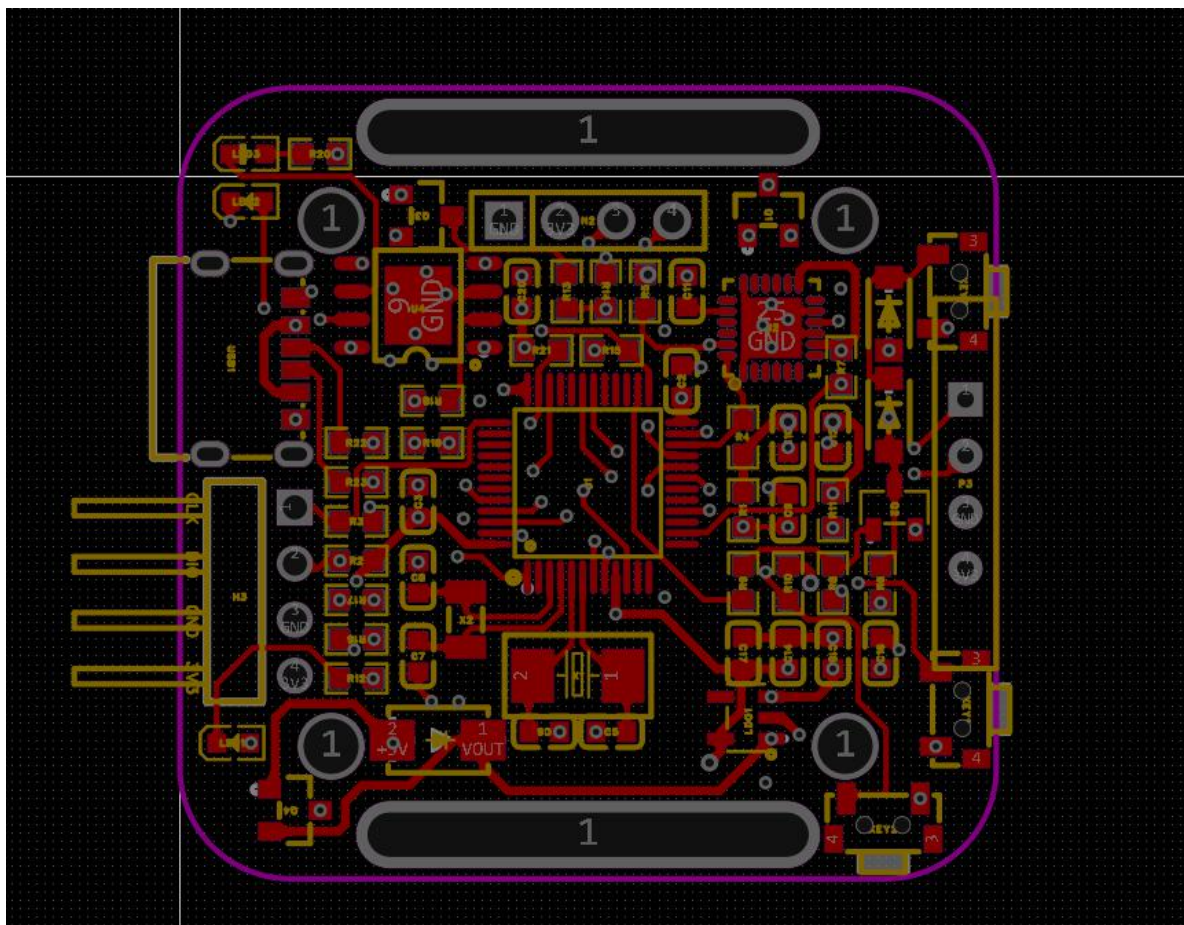
通过本项目，获得了宝贵的嵌入式系统开发经验：

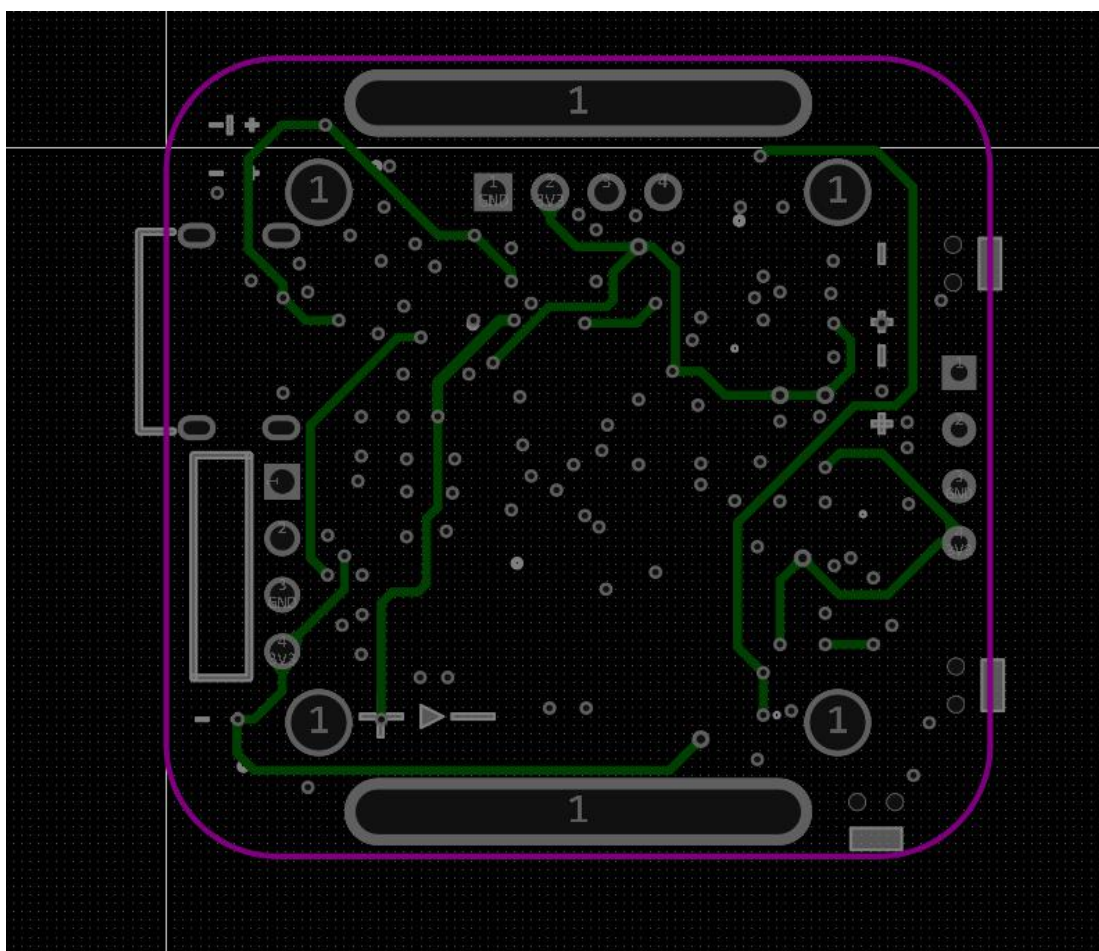
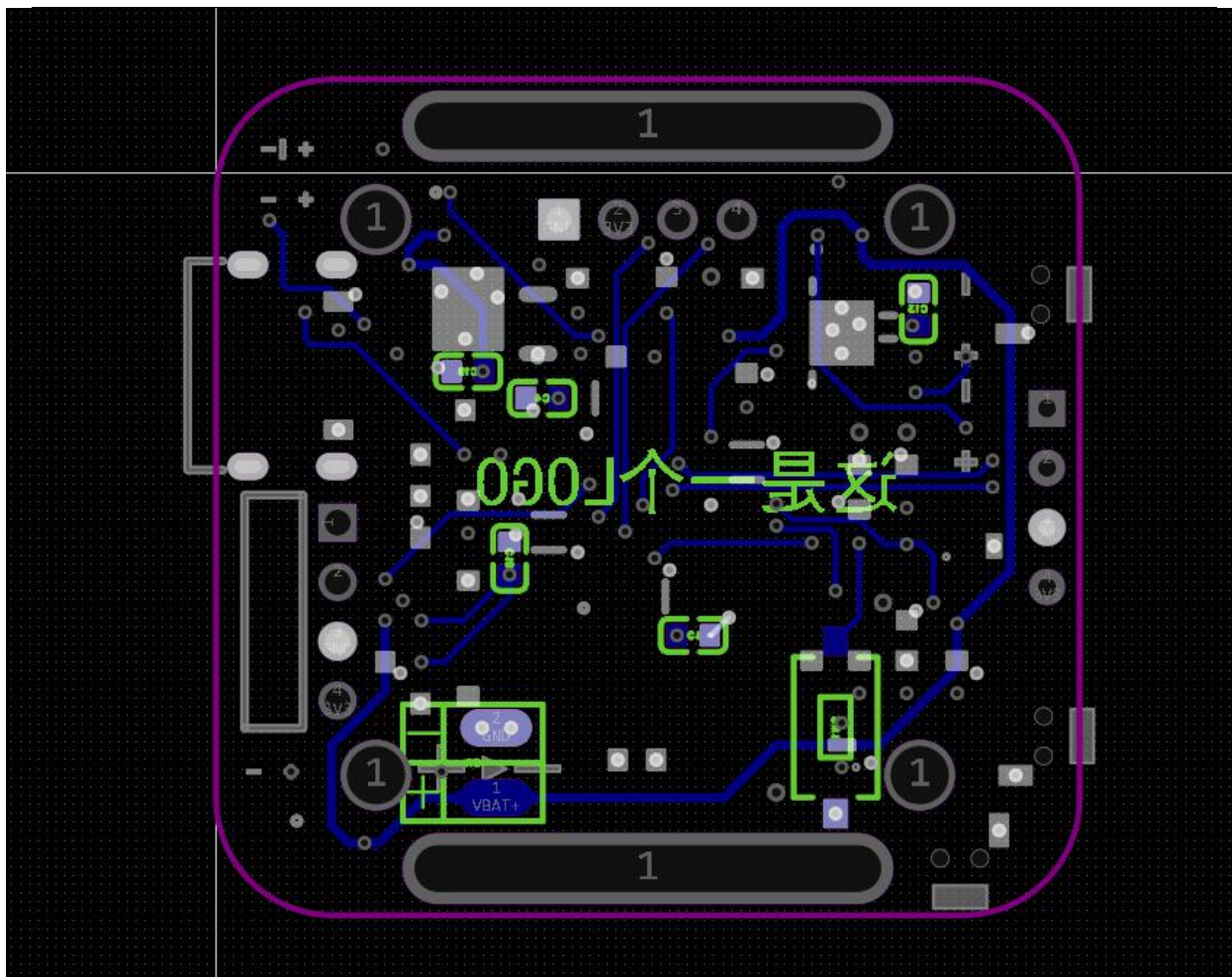
- 理论与实践结合：课堂学习的 USART、I2C、ADC等知识在实际项目中得到充分应用
- 成本意识：认识到工程设计必须考虑物料成本和可量产性
- 技术视野拓展：认识到物联网技术在智能电子中的重要性，为后续学习指明方向

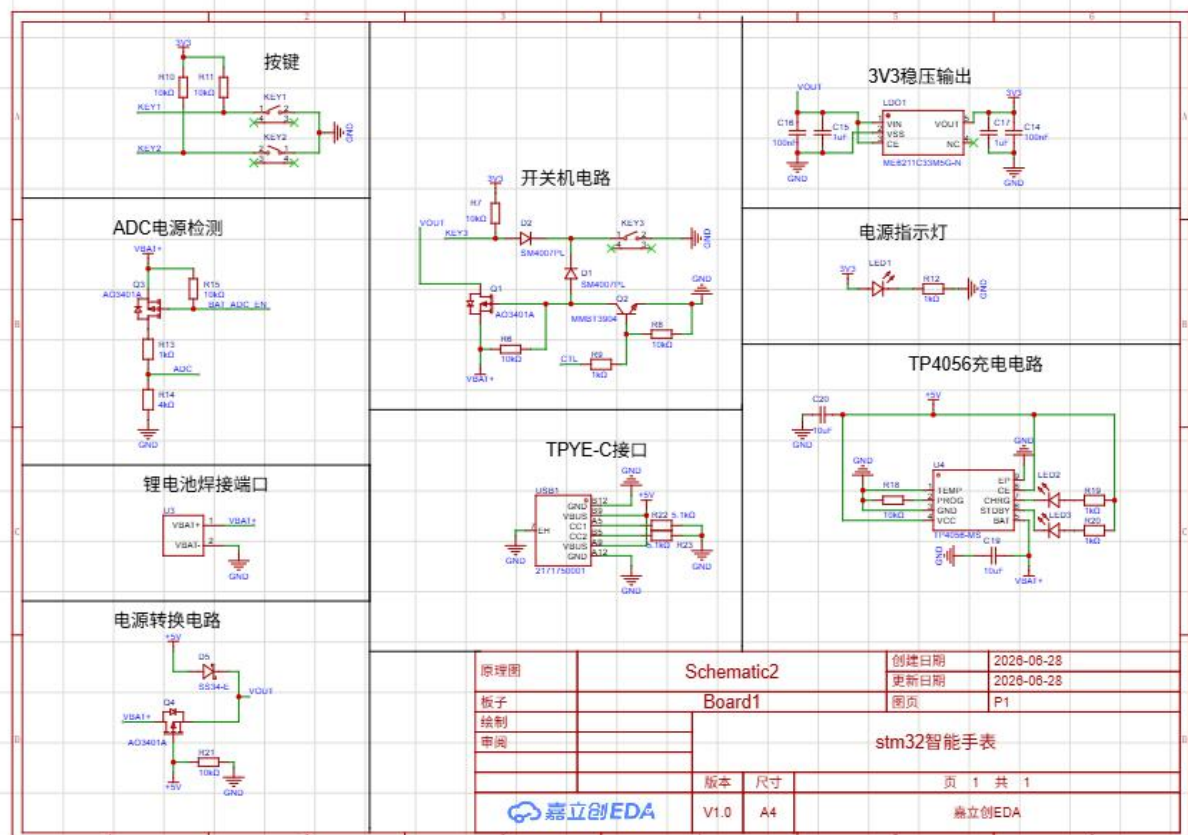
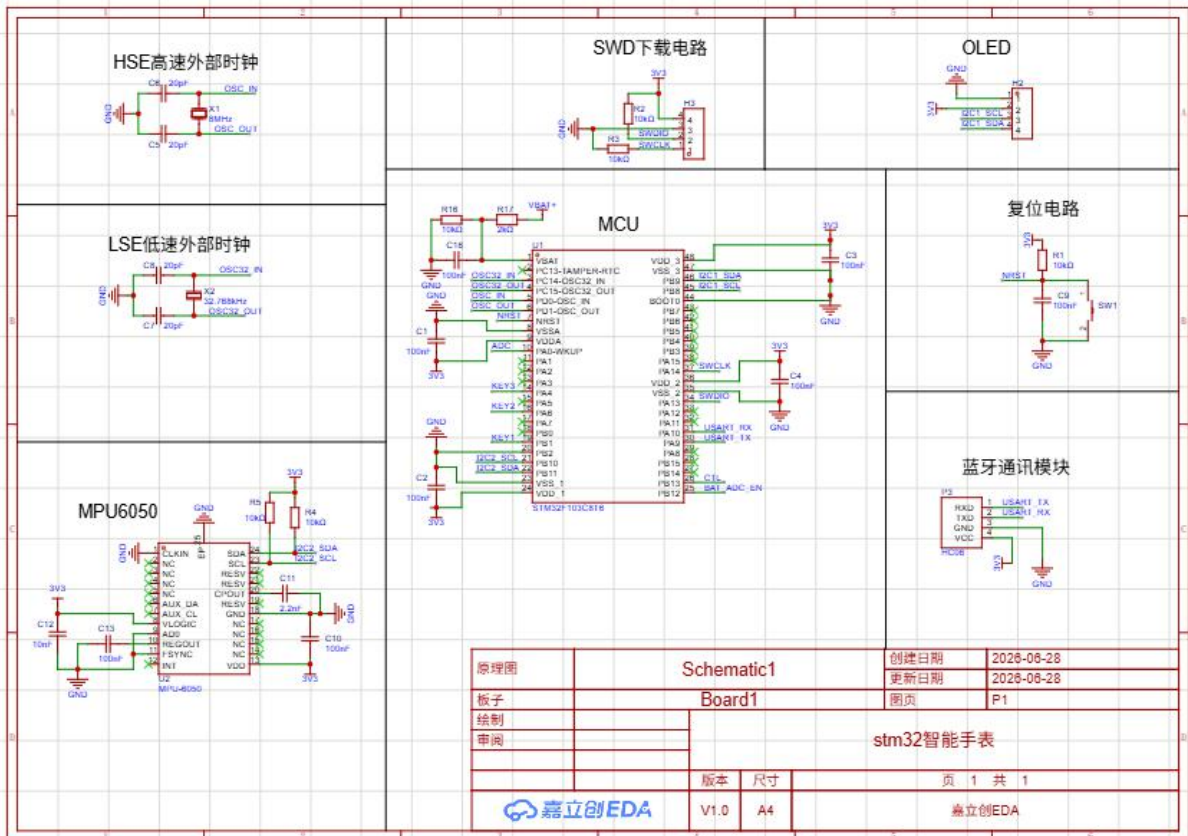
未来改进方向：

1. 修改为触摸屏显示
2. 增添心率检测功能
3. 增添温度传感功能
4. 设计外壳

A 设计图纸







```

1 #include "stm32f10x.h"           // Device header
2 #include "OLED.h"
3 #include "menu.h"
4 #include "SetTime.h"
5 #include "MPU6050.h"
6 #include "FreeRTOS.h"
7 #include "task.h"
8
9 /* 任务句柄 */
10 static TaskHandle_t UITaskHandle = NULL;
11 static TaskHandle_t SensorTaskHandle = NULL;
12
13 /*
14  * UI 任务（优先级 2）：运行界面状态机
15  * 时钟首页 → (菜单 | 设置) → 各应用(计步/游戏/时间设置/电量) → 返回
16  * 内部各界面函数都是阻塞式 while 循环，已在循环里加入 vTaskDelay 让步。
17  */
18 static void vUITask(void *pvParameters)
19 {
20     (void)pvParameters;
21     for(;;)
22     {
23         int page = First_Page_Clock(); //1=进入菜单, 2=进入设置
24         if(page==1)
25         {
26             Menu();
27         }
28         else if(page==2)
29         {
30             int s = SettingPage(); //返回 Setflag: 1=返回, 2=日期时间设置
31             if(s==2) SetTime();
32         }
33     }
34 }

```

